

Personalized Networking Assistant

Abstract of Project:

The Personalized Networking Assistant is an AI-driven web application that helps professionals generate context-aware conversation starters for networking events. Leveraging DistilBERT for zero-shot theme extraction and GPT-2 for natural language generation, the system analyzes event descriptions and user interests to produce personalized, engaging icebreakers. Built with FastAPI and Streamlit, it offers real-time Wikipedia-based fact-checking, user feedback mechanisms, session history logging, and a clean, modular architecture. This project demonstrates expertise in Generative AI integration, full-stack Python development, RESTful API design, and comprehensive testing with over 80% test coverage.

Problem Statement:

Professionals often struggle to initiate meaningful conversations at networking events due to lack of context, confidence, and personalized guidance. Generic conversation starters fail to engage effectively, and there is no integrated system that provides context-aware suggestions, real-time fact-checking, and progress tracking in a single platform. Existing approaches require manual research and rely on static icebreakers that do not adapt to individual interests or event themes. This project addresses these challenges by developing an AI-powered networking assistant that leverages DistilBERT for theme extraction, GPT-2 for personalized conversation generation, and the Wikipedia API for real-time fact-checking. The system aims to empower professionals to network with confidence, authenticity, and measurable improvement.

The Three Key Issues Are:

1. Lack of Personalization and Context

Professionals often rely on generic icebreakers that fail to engage effectively. There is no system that integrates individual interests, event themes, and real-time information to generate tailored, context-aware conversation starters.

2. Real Time Information Access

During conversations, professionals frequently need to verify facts or industry trends but lack immediate access to reliable sources. This leads to misinformation and reduced credibility.

3. No Progress Tracking or Feedback

There is no mechanism to track networking progress, review past interactions, or evaluate which conversation starters were effective, limiting opportunities for improvement.

The Core Features of the Personalized Networking Assistant:

1. AI-Powered Conversation Starter Generation

Generates personalized, context-aware conversation starters using GPT-2 based on user interests and event descriptions, helping users break the ice effortlessly.

2. Intelligent Theme Extraction

Uses DistilBERT to automatically extract key themes from event descriptions, ensuring conversation starters are relevant and aligned with the event context.

3. Real-Time Fact-Checking

Integrates with the Wikipedia API to verify facts, statistics, or claims instantly, helping users avoid misinformation and speak with confidence.

4. Personalized User Profiles

Allows users to create profiles with their bio, interests, and preferences, enabling the system to generate highly tailored conversation suggestions.

5. Session History Logging

Automatically logs all generated conversations, allowing users to revisit past interactions, track progress, and reflect on their networking journey.

6. Feedback System (Like/Dislike)

Enables users to provide instant feedback on conversation starters, helping the system learn preferences and improve future recommendations.

7. Interactive Frontend with Streamlit

Offers a clean, intuitive, and responsive user interface that makes it easy for users to input data, generate starters, and view results without any technical knowledge.

8. RESTful API Backend with FastAPI

Provides a robust and well-documented API that allows seamless communication between the frontend and backend, ensuring fast and reliable performance.

9. Conversation History View

Users can view their entire conversation history in one place, making it easy to track which starters worked best and prepare for future events.

10. Feedback Analytics Dashboard

Displays feedback statistics (total feedback, likes, dislikes, like rate) so users can understand which types of conversation starters are most effective.

Tech Stack Used in the Project:

1. Python 3.11+

The core programming language used to build the application backend, manage business logic, handle API requests, and run AI/NLP processing modules.

2. FastAPI 0.104.1

A modern, high-performance Python web framework used to create RESTful APIs with automatic interactive documentation (Swagger UI) and type-safe request validation.

3. Uvicorn 0.24.0

A lightning-fast ASGI server that serves the FastAPI application, handling concurrent requests efficiently with support for asynchronous operations.

4. Pydantic 2.4.2

A data validation library that uses Python type hints to validate request data, serialize responses, and ensure type safety across the application.

5. Streamlit 1.28.0

An open-source frontend framework that converts Python scripts into interactive web applications, providing a seamless user interface without requiring HTML, CSS, or JavaScript expertise.

6. Hugging Face Transformers 4.35.0

A powerful library providing thousands of pre-trained AI models, enabling seamless integration of DistilBERT for classification and GPT-2 for text generation.

7. DistilBERT (via Transformers)

A distilled, lightweight version of BERT used for zero-shot theme extraction, offering 60% faster inference while retaining 97% of BERT's performance.

8. GPT-2 (via Transformers)

A generative language model trained on 8 million web pages, used to produce natural, human-like conversation starters based on personalized prompts.

9. PyTorch 2.0.1+

An open-source deep learning framework that powers the Hugging Face models, handling tensor computations and neural network operations efficiently.

10. Accelerate 0.24.1

A library that simplifies running PyTorch models across different hardware (CPU/GPU), enabling faster model loading and optimized inference.

11. Wikipedia API 0.6.0

A Python wrapper for the Wikipedia REST API that enables real-time fact-checking by fetching article summaries and verifying information instantly.

12. Requests 2.31.0

A simple, elegant HTTP library used to communicate with external APIs, including Wikipedia, for fetching real-time data.

13. Pandas 2.1.3

A powerful data analysis and manipulation library used for processing structured data, managing feedback statistics, and handling JSON data.

14. NumPy 1.26.2

A fundamental library for numerical computing in Python, providing support for large, multi-dimensional arrays and mathematical operations.

15. Pytest 7.4.3

A robust testing framework that simplifies writing and executing unit tests, integration tests, and test-driven development workflows.

16. Pytest-Cov 4.1.0

A pytest plugin that measures code coverage, helping ensure at least 80% of the codebase is tested and reliable.

17. HTTPX 0.25.1

An async HTTP client used with FastAPI TestClient to test API endpoints without running a live server, ensuring fast and reliable integration tests.

18. Loguru 0.7.2

A beautiful and intuitive logging library that simplifies debugging, error tracking, and application monitoring with minimal configuration.

19. Python-Dotenv 1.0.0

A library that loads environment variables from a .env file, keeping sensitive information secure and configuration manageable across environments.

20. Git

A distributed version control system used for tracking changes, collaborating with team members, and maintaining code history throughout development.

21. Virtual Environment

A built-in Python tool that creates isolated environments for managing project dependencies separately, preventing version conflicts across projects.

22. Visual Studio Code

A lightweight but powerful source code editor with excellent Python support, integrated debugging, and Git integration for efficient development.

Structure Of The Project:

```
Personalized-Networking-Assistant/
|
├─ app/
|   ├── __init__.py
|   ├── main.py           # Application entry point
|   ├── config.py        # Configuration settings
|   |
|   ├── models/
|   |   ├── __init__.py
|   |   └── schemas.py    # Pydantic data models
|   |
|   ├── services/
|   |   ├── __init__.py
|   |   ├── event_analyzer.py # DistilBERT theme extraction
|   |   ├── topic_generator.py # GPT-2 conversation generation
|   |   ├── fact_checker.py   # Wikipedia API verification
|   |   ├── history_logger.py # Conversation history storage
|   |   └── feedback_logger.py # User feedback storage
|   |
|   └── routers/
|       ├── __init__.py
|       └── conversation.py   # API endpoints
|
├─ backend/                # Alternative location (you have this)
|   └── ... (your existing files)
|
├─ frontend/
|   └── app.py              # Streamlit UI
|
├─ data/                   # JSON storage
├─ requirements.txt
├─ setup.bat
└─ PROGRESS.md
```

ER DIAGRAM - PERSONALIZED NETWORKING ASSISTANT

The ER diagram illustrates the data structure for a Personalized Networking Assistant. It includes the following entities and their attributes:

- USER_PROFILE** (IMAGE 0+) Primary entity
 - Attributes: UserID, BioText, currentEventCache
- EVENT_CONTEXT** (IMAGE 0 & REFERENCE) The core event data
 - Attributes: EventID, EventDescription, AnalyzedThemes (Text)
- GENERATED_STARTER** (IMAGE 0 & REFERENCE) The resulting chat prompts
 - Attributes: StarterID, SessionID, StarterText, ContextPromptUsed
- LOG_ENTRY** (IMAGE 0 & REFERENCE) For auditing
 - Attributes: LogID, SessionID, ActionType (e.g., 'generate_starter'), PayloadJSON, Timestamp
- NETWORKING_SESSION** The core session log
 - Attributes: SessionID, UserID, EventID, SessionTimestamp
- WIKIPEDIA_FACT_CHECK** A supporting entity to trace fact checks
 - Attributes: FactCheckID, SessionID, VerifiedQueryText, VerificationStatus (e.g., verified, disputed), WikipediaSourceURL

Relationships:

- USER_PROFILE** (1) to **NETWORKING_SESSION** (m): One-to-many relationship.
- EVENT_CONTEXT** (1) to **NETWORKING_SESSION** (m): One-to-many relationship.
- GENERATED_STARTER** (1) to **NETWORKING_SESSION** (m): One-to-many relationship.
- LOG_ENTRY** (1) to **NETWORKING_SESSION** (m): One-to-many relationship.
- WIKIPEDIA_FACT_CHECK** (1) to **NETWORKING_SESSION** (m): One-to-many relationship.

Relationship Legend:

- 1:1 (Solid line)
- 1:m (Line with crow's foot notation)
- m:m (Line with crow's foot notation)

TECH STACK (IMAGE_4.PNG):

- Streamlit (UI), FastAPI (Backend), DistilBERT/GPT-2 (AI Models), Wikipedia API (Verification).

DATA PERSISTENCE APPROACH:

- Local JSON/CSV for User Profiles, Sessions, and Interaction Logs.

1. Create Project Folder Manually

cd Personalized-Networking-Assistant

Creating a virtual environment isolates project dependencies from your system Python.

- ### 3. Activate the Virtual Environment

Mac/Linux: source venv/bin/activate

Using requirements.txt

fastapi==0.104.1

```
uvicorn[standard]==0.24.0
streamlit==1.28.0
pydantic==2.4.2
pydantic-settings==2.0.3
transformers==4.35.0
torch>=2.0.0
torchvision>=0.15.0
torchaudio>=2.0.0
sentencepiece==0.1.99
accelerate==0.24.1
requests==2.31.0
httpx==0.25.1
python-dotenv==1.0.0
wikipedia-api==0.6.0
pandas==2.1.3
numpy==1.26.2
python-multipart==0.0.6
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-dateutil==2.8.2
fastapi-cors==0.0.6
loguru==0.7.2
pytest==7.4.3
pytest-cov==4.1.0
pytest-asyncio==0.21.1
```

Then install: **pip install -r requirements.txt**

5. Install PyTorch

PyTorch is required for running AI models.

For CPU (Recommended for most users):

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

For GPU (CUDA 11.8):

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118
```

For GPU (CUDA 12.1):

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu121
```

6. Verify Installation

Check if all packages are installed correctly:

```
python -c "import fastapi, streamlit, transformers, torch, requests, pandas; print('All  
packages installed successfully!')"
```

7. Create Project Directories

Create the necessary folders for the project:

Windows: mkdir app frontend tests data

Mac/Linux: mkdir -p app frontend tests data

8. Add Project Files

Add all the source files to their respective folders:

Backend Files (app/):

- app/main.py – FastAPI application entry point
- app/config.py – Configuration settings
- app/models/schemas.py – Pydantic data models
- app/services/event_analyzer.py – DistilBERT theme extraction
- app/services/topic_generator.py – GPT-2 conversation generation
- app/services/fact_checker.py – Wikipedia fact-checking
- app/services/history_logger.py – Conversation history
- app/services/feedback_logger.py – User feedback
- app/routers/conversation.py – API route definitions

Frontend Files (frontend/):

- frontend/streamlit_app.py – Streamlit UI

Test Files (tests/):

- tests/test_event_analyzer.py – Unit tests
- tests/test_topic_generator.py – Unit tests
- tests/test_fact_checker.py – Unit tests
- tests/test_api_routes.py – Integration tests

9. Set Up Data Directory

The application will create data/ folder automatically when running. This folder stores:

- history.json – Conversation history
- feedback.json – User feedback

10. Start the Backend Server

In Terminal 1: uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

Output:

- Application startup complete.
- Uvicorn running on <http://0.0.0.0:8000>

11. Start the Frontend Server

In Terminal 2 : Streamlit run frontend/streamlit_app.py

Output :

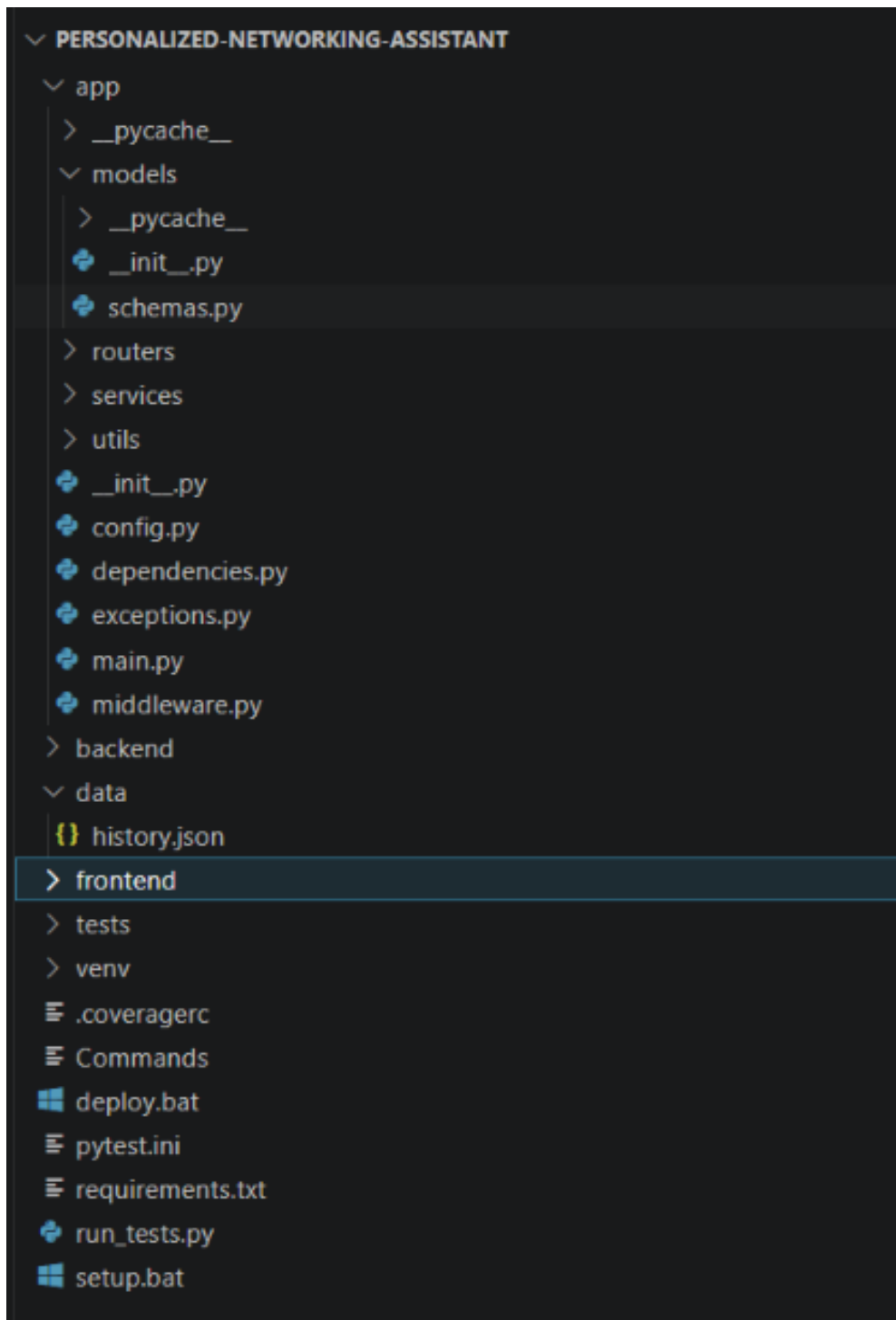
- You can now view your Streamlit app in your browser.
- Local URL: <http://localhost:8501>

12. Open in Browser

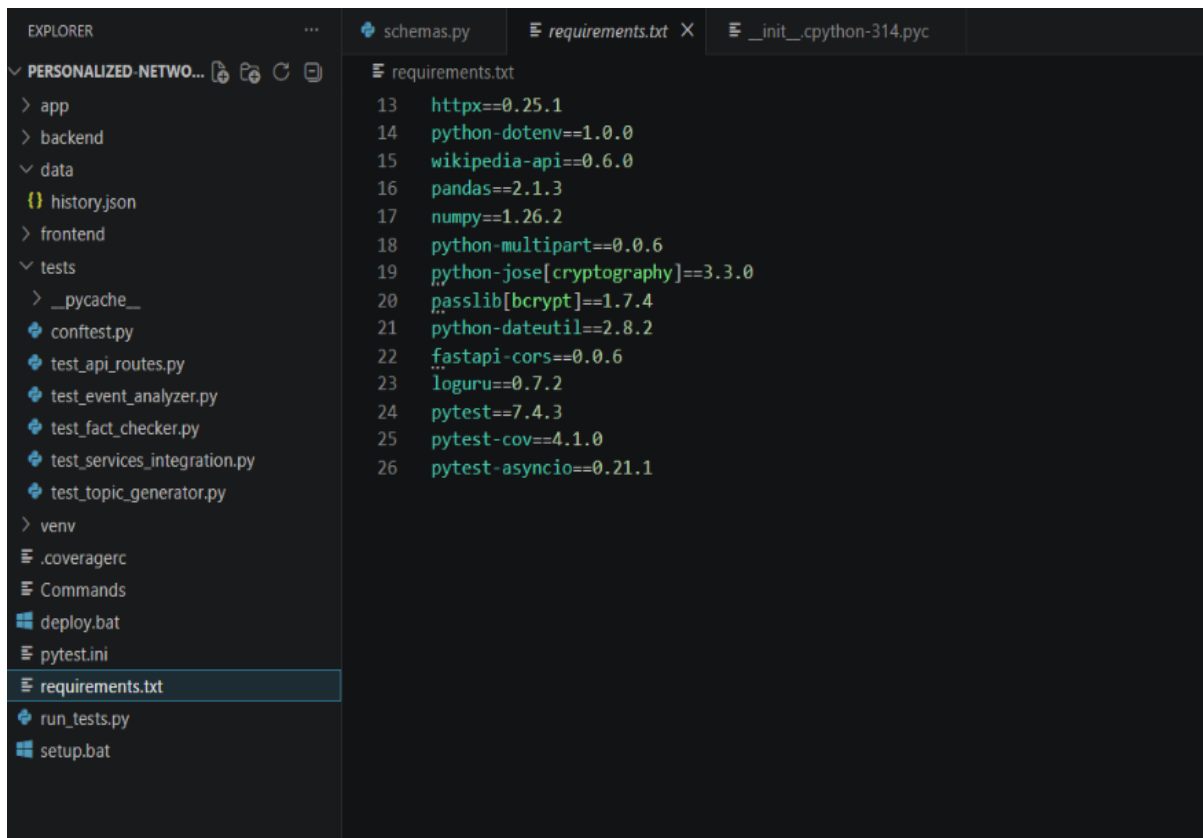
- Go to: <http://localhost:8501>

Screenshots Of The Project:

1. Defining the Application Architecture:



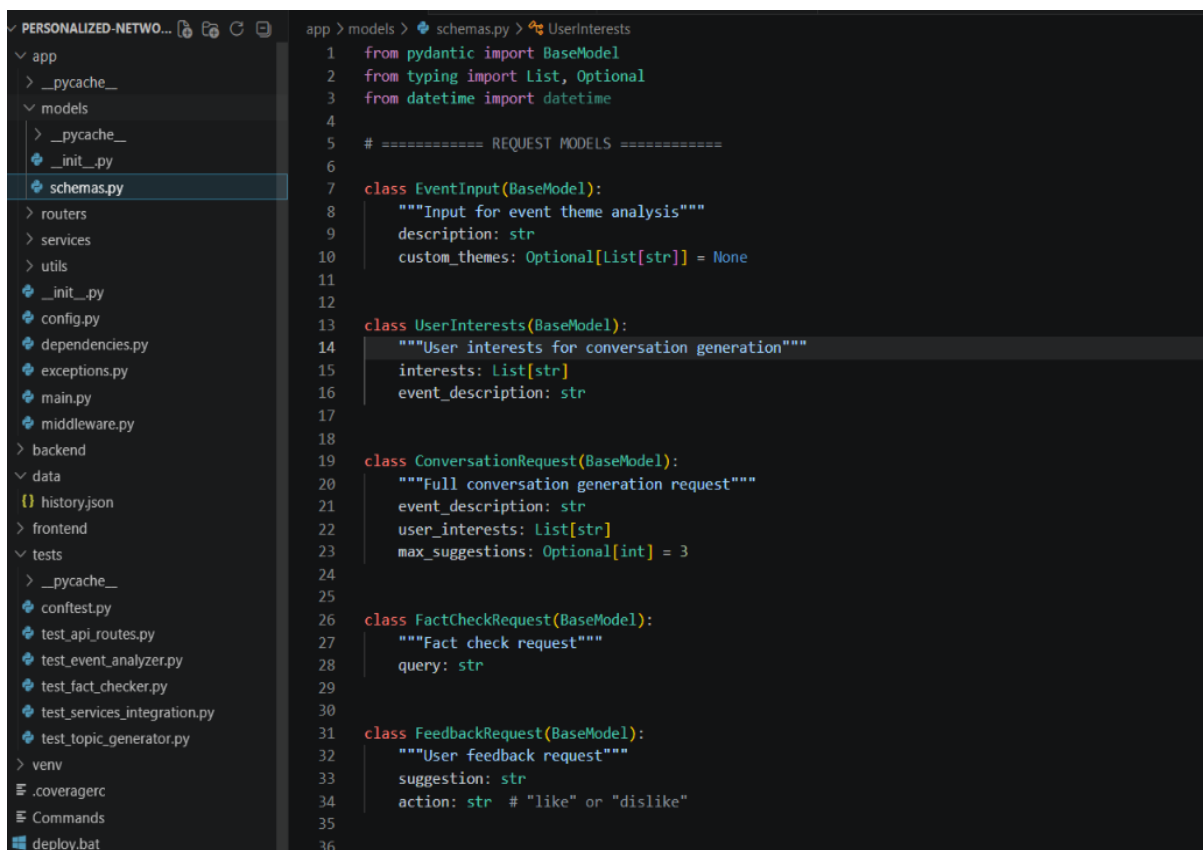
2. Environment Setup:



The screenshot shows the VS Code interface with the EXPLORER sidebar on the left. The file explorer is expanded to show the 'requirements.txt' file. The main editor area displays the contents of 'requirements.txt', which lists various Python dependencies and their versions.

```
requirements.txt
13 httpx==0.25.1
14 python-dotenv==1.0.0
15 wikipedia-api==0.6.0
16 pandas==2.1.3
17 numpy==1.26.2
18 python-multipart==0.0.6
19 python-jose[cryptography]==3.3.0
20 passlib[bcrypt]==1.7.4
21 python-dateutil==2.8.2
22 fastapi-cors==0.0.6
23 loguru==0.7.2
24 pytest==7.4.3
25 pytest-cov==4.1.0
26 pytest-asyncio==0.21.1
```

3. Data Schema Definition:



The screenshot shows the VS Code interface with the EXPLORER sidebar on the left. The file explorer is expanded to show the 'schemas.py' file. The main editor area displays the contents of 'schemas.py', which defines several Pydantic models for data schema.

```
schemas.py
1 from pydantic import BaseModel
2 from typing import List, Optional
3 from datetime import datetime
4
5 # ===== REQUEST MODELS =====
6
7 class EventInput(BaseModel):
8     """Input for event theme analysis"""
9     description: str
10     custom_themes: Optional[List[str]] = None
11
12
13 class UserInterests(BaseModel):
14     """User interests for conversation generation"""
15     interests: List[str]
16     event_description: str
17
18
19 class ConversationRequest(BaseModel):
20     """Full conversation generation request"""
21     event_description: str
22     user_interests: List[str]
23     max_suggestions: Optional[int] = 3
24
25
26 class FactCheckRequest(BaseModel):
27     """Fact check request"""
28     query: str
29
30
31 class FeedbackRequest(BaseModel):
32     """User feedback request"""
33     suggestion: str
34     action: str # "like" or "dislike"
35
36
```

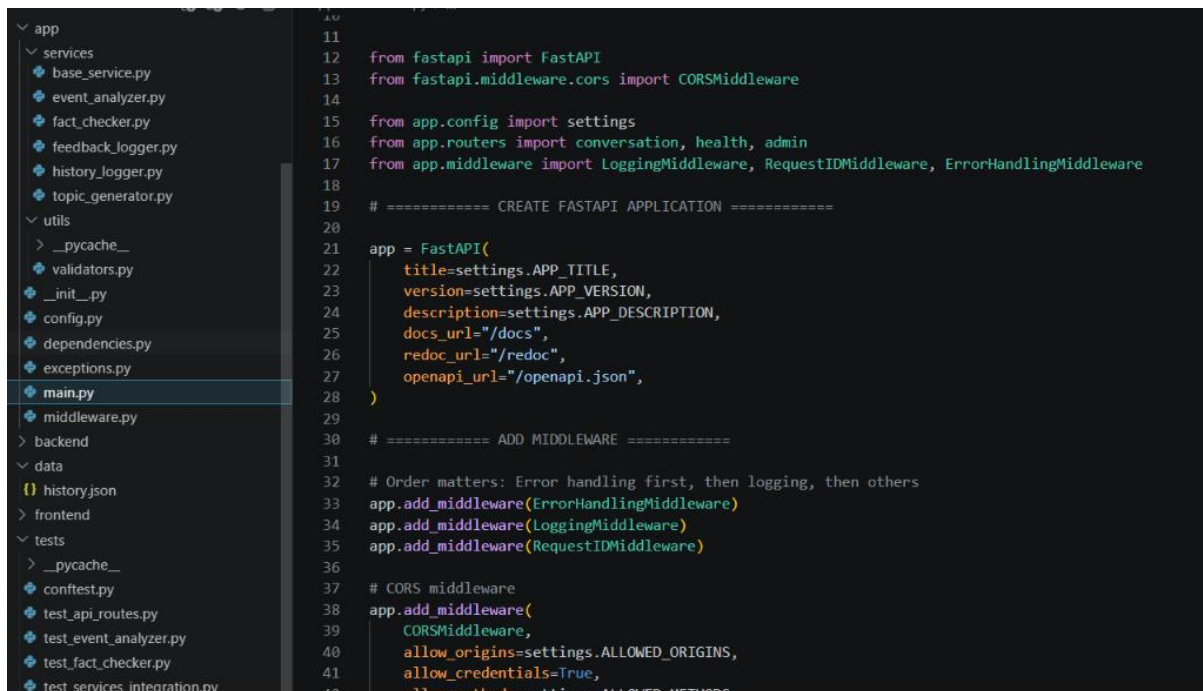
4. Topic Generator Service:

```
PERSONALIZED-NETWO...  app > services > topic_generator.py > ...
1 """
2 Topic Generator Service - Conversation Generation using GPT-2
3 Based on your documentation: Epic 2 - Topic Generator Service
4 """
5
6 from transformers import pipeline, set_seed
7 from typing import List
8 import random
9
10 # Set seed for reproducibility
11 set_seed(42)
12
13 # Load model once at startup
14 print("Loading GPT-2 model for conversation generation...")
15 generator = pipeline(
16     "text-generation",
17     model="gpt2",
18     max_length=80
19 )
20 print("✅ GPT-2 loaded successfully!")
21
22
23 def generate_topics(
24     themes: List[str],
25     user_interests: List[str],
26     max_suggestions: int = 3
27 ) -> List[str]:
28     """
29     Generate conversation starters using GPT-2
30
31     Args:
32         themes: Extracted event themes
33         user_interests: User's interests/interests list
```

5. Feedback Logger Service:

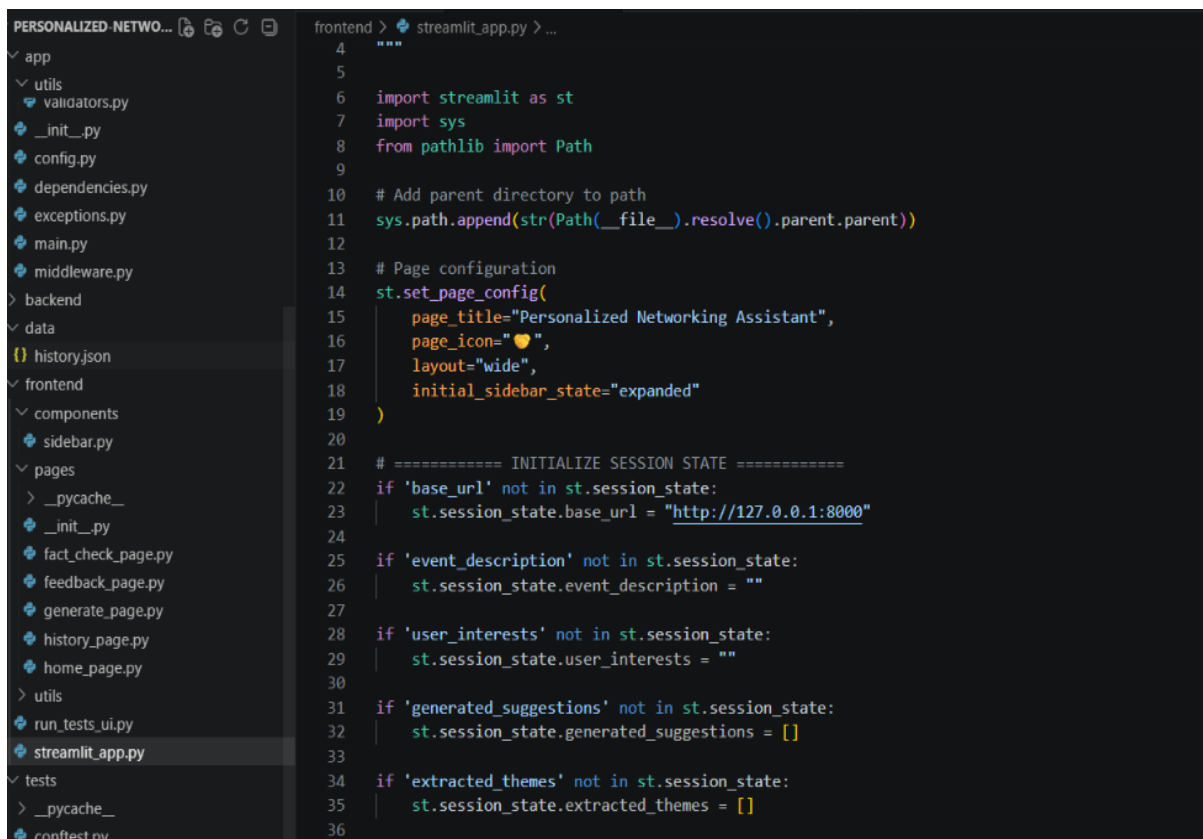
```
PERSONALIZED-NETWO...  app > services > feedback_logger.py > ...
1 """
2 Feedback Logger Service - User Feedback Collection
3 Based on your documentation: Epic 2 - Feedback Logger Service
4 """
5
6 import json
7 from datetime import datetime
8 from pathlib import Path
9 from typing import List, Dict, Optional
10
11 from app.config import FEEDBACK_FILE
12
13
14 def log_feedback(suggestion: str, action: str) -> Dict:
15     """
16     Save user feedback for a suggestion
17
18     Args:
19         suggestion: The suggestion text
20         action: 'like' or 'dislike'
21
22     Returns:
23         The feedback entry
24     """
25     entry = {
26         "suggestion": suggestion,
27         "feedback": action,
28         "timestamp": datetime.now().isoformat()
29     }
30
31     # Load existing feedback
```

6. Application Entry Point:



```
11
12 from fastapi import FastAPI
13 from fastapi.middleware.cors import CORSMiddleware
14
15 from app.config import settings
16 from app.routers import conversation, health, admin
17 from app.middleware import LoggingMiddleware, RequestIDMiddleware, ErrorHandlingMiddleware
18
19 # ===== CREATE FASTAPI APPLICATION =====
20
21 app = FastAPI(
22     title=settings.APP_TITLE,
23     version=settings.APP_VERSION,
24     description=settings.APP_DESCRIPTION,
25     docs_url="/docs",
26     redoc_url="/redoc",
27     openapi_url="/openapi.json",
28 )
29
30 # ===== ADD MIDDLEWARE =====
31
32 # Order matters: Error handling first, then logging, then others
33 app.add_middleware(ErrorHandlingMiddleware)
34 app.add_middleware(LoggingMiddleware)
35 app.add_middleware(RequestIDMiddleware)
36
37 # CORS middleware
38 app.add_middleware(
39     CORSMiddleware,
40     allow_origins=settings.ALLOWED_ORIGINS,
41     allow_credentials=True,
42     allow_methods=settings.ALLOWED_METHODS,
```

7. Application Setup and Configuration:



```
frontend > streamlit_app.py > ...
4
5
6 import streamlit as st
7 import sys
8 from pathlib import Path
9
10 # Add parent directory to path
11 sys.path.append(str(Path(__file__).resolve().parent.parent))
12
13 # Page configuration
14 st.set_page_config(
15     page_title="Personalized Networking Assistant",
16     page_icon="❤️",
17     layout="wide",
18     initial_sidebar_state="expanded"
19 )
20
21 # ===== INITIALIZE SESSION STATE =====
22 if 'base_url' not in st.session_state:
23     st.session_state.base_url = "http://127.0.0.1:8000"
24
25 if 'event_description' not in st.session_state:
26     st.session_state.event_description = ""
27
28 if 'user_interests' not in st.session_state:
29     st.session_state.user_interests = ""
30
31 if 'generated_suggestions' not in st.session_state:
32     st.session_state.generated_suggestions = []
33
34 if 'extracted_themes' not in st.session_state:
35     st.session_state.extracted_themes = []
36
```

8. Testing and Local Deployment:

```
PERSONALIZED-NETWO... [F] [G] [C] [D]
> backend
> data
  {} history.json
> frontend
  > components
    sidebar.py
  > pages
    > _pycache_
    _init_.py
    fact_check_page.py
    feedback_page.py
    generate_page.py
    history_page.py
    home_page.py
  > utils
    run_tests_ui.py
    streamlit_app.py
  > tests
    > _pycache_
    conftest.py
    test_api_routes.py
    test_event_analyzer.py
    test_fact_checker.py
    test_services_integration.py
    test_topic_generator.py
  > venv
  .coveragerc
  Commands

tests > test_api_routes.py > ...
/
8
9 import pytest
10 from fastapi.testclient import TestClient
11 from app.main import app
12
13 client = TestClient(app)
14
15
16 class TestAPIRoutes:
17     """Test suite for API routes"""
18
19     # ===== HEALTH ROUTES =====
20
21     def test_health_check(self):
22         """Test health check endpoint"""
23         response = client.get("/health")
24         assert response.status_code == 200
25         data = response.json()
26         assert data["status"] == "healthy"
27         assert "service" in data
28
29     def test_root_endpoint(self):
30         """Test root endpoint"""
31         response = client.get("/")
32         assert response.status_code == 200
33         data = response.json()
34         assert "message" in data
35         assert "Welcome" in data["message"]
36
37     # ===== EVENT ANALYZER ROUTE =====
38
```

```
PERSONALIZED-NETWO... [F] [G] [C] [D]
> backend
> data
  {} history.json
> frontend
  > components
    sidebar.py
  > pages
    > _pycache_
    _init_.py
    fact_check_page.py
    feedback_page.py
    generate_page.py
    history_page.py
    home_page.py
  > utils
    run_tests_ui.py
    streamlit_app.py
  > tests
    > _pycache_
    conftest.py
    test_api_routes.py
    test_event_analyzer.py
    test_fact_checker.py
    test_services_integration.py
    test_topic_generator.py
  > venv
  .coveragerc
  Commands
  deploy.bat
  pytest.ini
  requirements.txt
  run_tests.py

tests > test_fact_checker.py > ...
/
8
9 import pytest
10 from unittest.mock import patch, MagicMock
11 from app.services import fact_checker
12
13
14 class TestFactChecker:
15     """Test suite for fact checker service"""
16
17     @patch('app.services.fact_checker.requests.get')
18     def test_fact_check_returns_dict(self, mock_get, sample_fact_query):
19         """Test that fact_check returns a dictionary"""
20         # Mock successful response
21         mock_response = MagicMock()
22         mock_response.status_code = 200
23         mock_response.json.return_value = {
24             'query': {
25                 'search': [{'title': 'Artificial Intelligence'}]
26             }
27         }
28         mock_get.return_value = mock_response
29
30         result = fact_checker.fact_check(sample_fact_query)
31         assert isinstance(result, dict)
32         assert 'query' in result
33         assert 'summary' in result
34         assert 'verified' in result
35
36     @patch('app.services.fact_checker.requests.get')
37     def test_fact_check_returns_summary_string(self, mock_get, sample_fact_query):
38         """Test that fact_check returns a summary string"""
39         mock_response = MagicMock()
40         mock_response.status_code = 200
41         mock_response.json.side_effect = [
42             ({'query': {'search': [{'title': 'Artificial Intelligence'}]}},
43              {'query': {'pages': {'1': {'extract': 'AI is a field...'}}})
44         ]
45
```

9. Manual Testing and Verification:

- **Backend Images(Terminal 1):**

Command: `uvicorn app.main:app --reload --host 0.0.0.0 --port 8000`

```
PS D:\Internship Projects\Personalized-Networking-Assistant> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ;  
Networking-Assistant\venv\Scripts\Activate.ps1")  
(venv) PS D:\Internship Projects\Personalized-Networking-Assistant> uvicorn app.main:app --reload --host 0.0.0.0 --port 8000  
INFO: Will watch for changes in these directories: ['D:\\Internship Projects\\Personalized-Networking-Assistant']
```

- **Frontend(Terminal 2):**

Command: streamlit run frontend/streamlit app.py

```
PS D:\Internship Projects\Personalized-Networking-Assistant> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "d:\Internship Projects\Personalized-Networking-Assistant\venv\Scripts\Activate.ps1")
(venv) PS D:\Internship Projects\Personalized-Networking-Assistant> streamlit run frontend/streamlit_app.py
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Internship\Projects\Personalized-Networking-Assistant> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "d:\Internship\Projects\Personalized-Networking-Assistant\venv\Scripts\Activate.ps1")
(venv) PS D:\Internship\Projects\Personalized-Networking-Assistant> streamlit run frontend\streamlit_app.py
2026-07-04 13:37:05.001 Unicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.103:8501

```

Output:

